

APOSTILA PRÁTICA: DESENVOLVIMENTO DE INTERFACE GRÁFICA COM PYSIDE6 E QT DESIGNER

Projeto de Aplicação: Horizon Media Player

Autor: Wagner Rodrigues da Silva

Público-alvo: Estudantes e Profissionais de TI / Desenvolvimento de Software

Tecnologias: Python 3.10+, PySide6 (Qt for Python), Qt Designer, PyCharm

1. INTRODUÇÃO E ARQUITETURA DE SOFTWARE

A criação de aplicações gráficas modernas exige uma clara separação entre a **Camada de Apresentação (Interface do Usuário - UI)** e a **Camada de Lógica de Negócios (Regras de Aplicação e Controle)**. No ecossistema Python, o **PySide6** é a biblioteca oficial do framework Qt, oferecendo alto desempenho, portabilidade multiplataforma e um conjunto robusto de ferramentas de design.

O Paradigma da Separação de Responsabilidades

Ao desenvolver interfaces ricas, escrever cada botão e layout manualmente via código puro em Python pode se tornar contraproducente e difícil de manter. A abordagem profissional consiste em:

1. **Design Visual Declarativo:** Criar a interface visual no **Qt Designer**, salvando a estrutura em formato XML (**.ui**).
2. **Compilação de Recursos:** Converter o arquivo **.ui** para uma classe Python intermediária (**interface_ui.py**) usando o compilador oficial **pyside6-uic**.
3. **Controlador e Lógica de Negócios:** No arquivo **main.py**, herdar os componentes visuais e conectar os eventos (*Signals & Slots*) ao motor da aplicação (**QMediaPlayer** e **QAudioOutput**).

2. PREPARAÇÃO DO AMBIENTE NO PYCHARM

2.1. Criação do Projeto e Ambiente Virtual

1. Abra o PyCharm e crie um novo projeto com interpretador virtual (**venv**).
2. Abra o terminal integrado do PyCharm e instale o PySide6:

```
pip install pyside6
```

2.2. Acesso ao Qt Designer

O executável do Qt Designer já acompanha a instalação do PySide6. Para iniciá-lo, digite no terminal do PyCharm: `pyside6-designer`

(Dica de Produtividade: No PyCharm, é possível configurar o Qt Designer em File > Settings > Tools > External Tools para abri-lo com um clique direito no projeto).

3. PROJETANDO A INTERFACE NO QT DESIGNER

3.1. Criação da Janela Base

1. Na janela de diálogo do Qt Designer, selecione o template **Main Window** e clique em **Create**.
2. No menu lateral direito (Property Editor), altere a propriedade `objectName` da janela para `MainWindow`.

3.2. Estrutura de Widgets Necessários

Adicione os seguintes componentes à janela, definindo atentamente o `objectName` de cada um (eles serão os identificadores no código Python):

Componente (Widget)	objectName	Finalidade Pedagógica
QLabel ▾	label_musica	Exibe o nome da faixa em reprodução ou status inicial.
QListWidget ▾	lista_musicas	Lista de faixas com barra de rolagem (scroll) nativa.
QSlider (Horizontal) ▾	slider_posicao	Linha do tempo interativa da música (progresso/seek).
QLabel ▾	label_tempo_atual	Exibe o tempo decorrido no formato 00:00.
QLabel ▾	label_tempo_total	Exibe a duração total no formato 00:00.
QPushButton ▾	btn_abrir	Abre a caixa de diálogo para carregar múltiplos arquivos.
QPushButton ▾	btn_retroceder	Retrocede 10 segundos da reprodução atual.
QPushButton ▾	btn_play	Inicia/retoma a reprodução da música.
QPushButton ▾	btn_pause	Pausa a música sem perder a posição.
QPushButton ▾	btn_parar	Interrompe a música e reinicia a posição.

Componente (Widget)	objectName	Finalidade Pedagógica
QPushButton ▾	<code>btn_avancar</code>	Avança 10 segundos na faixa.
QSlider (Horizontal) ▾	<code>slider_volume</code>	Controle de nível de ganho de áudio (0 a 100%).

3.3. Salvando o Arquivo

Salve o trabalho na pasta raiz do projeto com o nome `interface.ui`.

4. COMPILAÇÃO DO ARQUIVO .UI PARA PYTHON

Para que o Python consiga instanciar os elementos criados no Qt Designer sem custo de interpretação em tempo de execução, compilamos o arquivo `.ui` para Python executando o utilitário `pyside6-uic`: `pyside6-uic interface.ui -o interface_ui.py`

Regra de Ouro: Nunca altere diretamente o arquivo `interface_ui.py`. Se precisar modificar o layout, abra o `interface.ui` no Qt Designer, faça as alterações, salve e execute o comando de compilação novamente.

5. CÓDIGO FONTE COMPLETO E DIDÁTICO (main.py)

Crie o arquivo `main.py` na raiz do projeto com a implementação abaixo. Todo o código está detalhadamente comentado para facilitar a explicação em sala de aula:

```
import sys
```

```
# Importações dos módulos centrais do PySide6
```

```
from PySide6.QtCore import Qt, QUrl
```

```
from PySide6.QtMultimedia import QAudioOutput, QMediaPlayer
```

```
from PySide6.QtWidgets import (
```

```
    QApplication,
```

```
    QFileDialog,
```

```
    QMainWindow,
```

```
    QStyle,
```

```
)
```

```
# Importação da interface gerada pelo comando pyside6-uic
```

```
from interface_ui import Ui_MainWindow
```

```
class HorizonPlayer(QMainWindow):
```

```
    """
```

```
    Classe controladora do Horizon Media Player.
```

```
    Herda de QMainWindow para fornecer a estrutura padrão de janelas do sistema operacional.
```

```
    """
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        # Instancia e injeta os componentes visuais do Qt Designer na janela
```

```
self.ui = Ui_MainWindow()

self.ui.setupUi(self)

# Configura o título da janela principal

self.setWindowTitle(" 🎵 Horizon Media Player")

# Estrutura de dados para gerenciar as faixas carregadas

# Armazena dicionários: {"nome": str, "caminho": str}

self.playlist = []

self.indice_atual = -1

# ----- NÚCLEO DE MULTIMÍDIA DO QT -----

# QMediaPlayer: Gerencia o fluxo de controle de mídia (Play, Pause, Stop,
Seek)

self.player = QMediaPlayer(self)

# QAudioOutput: Gerencia a comunicação com o hardware de som do
sistema operacional

self.audio_output = QAudioOutput(self)

self.player.setAudioOutput(self.audio_output)

# Define o volume inicial em 50% (faixa aceita: 0.0 a 1.0)

self.audio_output.setVolume(0.5)

# Configura os ícones e propriedades dos widgets
```

```
self._configurar_interface()

# Estabelece as conexões entre eventos de interface e métodos da classe
(Signals & Slots)

self._conectar_sinais()

def _configurar_interface(self):

    """Aplica ícones padrão do sistema operacional e define os limites dos
    sliders."""

    estilo = self.style()

    # Atribuição de ícones padronizados aos botões de controle

    self.ui.btn_abrir.setIcon(estilo.standardIcon(QStyle.SP_DirOpenIcon))

self.ui.btn_retroceder.setIcon(estilo.standardIcon(QStyle.SP_MediaSeekBackward)
)

self.ui.btn_play.setIcon(estilo.standardIcon(QStyle.SP_MediaPlay))

self.ui.btn_pause.setIcon(estilo.standardIcon(QStyle.SP_MediaPause))

self.ui.btn_parar.setIcon(estilo.standardIcon(QStyle.SP_MediaStop))

self.ui.btn_avancar.setIcon(estilo.standardIcon(QStyle.SP_MediaSeekForward))

# Configuração do slider de volume (escala inteira de 0 a 100)

self.ui.slider_volume.setRange(0, 100)
```

```
self.ui.slider_volume.setValue(50)

# Configuração inicial da barra de progresso

self.ui.slider_posicao.setRange(0, 0)

def _conectar_sinais(self):

    """Vincula os sinais emitidos pelos widgets aos seus respectivos métodos
    tratadores (Slots)."""

    # Eventos de clique nos botões

    self.ui.btn_abrir.clicked.connect(self.abrir_arquivos)

    self.ui.btn_play.clicked.connect(self.player.play)

    self.ui.btn_pause.clicked.connect(self.player.pause)

    self.ui.btn_parar.clicked.connect(self.player.stop)

    self.ui.btn_avancar.clicked.connect(self.avancar_10s)

    self.ui.btn_retroceder.clicked.connect(self.retroceder_10s)

    # Duplo clique na lista de músicas (QListWidget)

self.ui.lista_musicas.itemDoubleClicked.connect(self.tocar_musica_selecionada)

# Controle de volume: Converte valor inteiro (0-100) para float (0.0-1.0)

self.ui.slider_volume.valueChanged.connect(

    lambda v: self.audio_output.setVolume(v / 100.0)
```



```

)

# Progresso da faixa arrastado pelo usuário na barra

self.ui.slider_posicao.sliderMoved.connect(self.player.setPosition)

# Sinais emitidos pelo próprio QMediaPlayer para atualizar a UI em tempo
real

self.player.positionChanged.connect(self.atualizar_posicao)

self.player.durationChanged.connect(self.atualizar_duracao)

self.player.mediaStatusChanged.connect(self.verificar_fim_da_faixa)

# ===== MÉTODOS DE CONTROLE
=====

def abrir_arquivos(self):

    """Abre janela de diálogo do sistema permitindo seleção de múltiplos
    arquivos de áudio."""

    arquivos, _ = QFileDialog.getOpenFileNames(

        self,

        "Selecionar Músicas para o Horizon",

        "",

        "Arquivos de Áudio (*.mp3 *.wav *.ogg *.flac *.m4a)",

    )

    if arquivos:

```

```
for caminho in arquivos:

    nome_arquivo = os.path.basename(caminho)

    # Adiciona o item à lista interna

    self.playlist.append({"nome": nome_arquivo, "caminho": caminho})

    # Adiciona o elemento visual na QListWidget (com suporte a rolagem automática)

    self.ui.lista_musicas.addItem(f" 🎵 {nome_arquivo}")

    # Se nenhuma faixa estiver em reprodução, carrega e inicia a primeira automaticamente

    if self.indice_atual == -1 and len(self.playlist) > 0:

        self.carregar_e_tocar(0)

def carregar_e_tocar(self, indice):

    """Carrega a música especificada pelo índice no QMediaPlayer e inicia o playback."""

    if 0 <= indice < len(self.playlist):

        self.indice_atual = indice

        faixa = self.playlist[indice]

        # O QMediaPlayer requer uma QUrl válida para carregar o arquivo local

        url_local = QUrl.fromLocalFile(faixa["caminho"])

        self.player.setSource(url_local)
```

```

# Atualiza o rótulo de texto na interface gráfica

self.ui.label_musica.setText(f"🎵 Tocando: {faixa['nome']}")

# Destaca a linha correspondente na QListWidget

self.ui.lista_musicas.setCurrentRow(indice)

# Inicia a execução do som

self.player.play()

def tocar_musica_selecionada(self):

    """Acionado ao dar duplo clique em um item da lista visual."""

    linha_selecionada = self.ui.lista_musicas.currentRow()

    if linha_selecionada >= 0:

        self.carregar_e_tocar(linha_selecionada)

def avancar_10s(self):

    """Avança a reprodução em 10.000 milissegundos (10 segundos)."""

    nova_pos = self.player.position() + 10_000

    self.player.setPosition(min(nova_pos, self.player.duration()))

def retroceder_10s(self):

    """Retrocede a reprodução em 10.000 milissegundos (10 segundos)."""

    nova_pos = self.player.position() - 10_000

```

```
self.player.setPosition(max(nova_pos, 0))

def atualizar_posicao(self, posicao_ms):

    """Atualiza a posição do slider e o contador de tempo decorrido."""

    # Evita oscilação na barra enquanto o usuário estiver arrastando com o
mouse

    if not self.ui.slider_posicao.isSliderDown():

        self.ui.slider_posicao.setValue(posicao_ms)

        self.ui.label_tempo_atual.setText(self.formatar_tempo(posicao_ms))

def atualizar_duracao(self, duracao_ms):

    """Define o valor máximo do slider quando a duração total do arquivo é
reconhecida."""

    self.ui.slider_posicao.setRange(0, duracao_ms)

    self.ui.label_tempo_total.setText(self.formatar_tempo(duracao_ms))

def verificar_fim_da_faixa(self, status):

    """Avança automaticamente para a próxima música ao final da faixa atual."""

    if status == QMediaPlayer.MediaStatus.EndOfMedia:

        proximo_indice = self.indice_atual + 1

        if proximo_indice < len(self.playlist):

            self.carregar_e_tocar(proximo_indice)
```

```

        else:

            # Fim da lista atingido

            self.player.stop()

    @staticmethod

    def formatar_tempo(milissegundos):

        """Função utilitária estática para converter milissegundos no formato textual
        MM:SS."""

        segundos = milissegundos // 1000

        minutos = segundos // 60

        segundos_restantes = segundos % 60

        return f"{minutos:02d}:{segundos_restantes:02d}"

# ===== EXECUÇÃO DA APLICAÇÃO
=====

if __name__ == "__main__":

    # Inicializa o loop de eventos da aplicação Qt

    app = QApplication(sys.argv)

    # Carrega folha de estilo customizada (style.qss) caso exista

    if os.path.exists("style.qss"):

        with open("style.qss", "r", encoding="utf-8") as f:

```

```
app.setStyleSheet(f.read())

# Cria e exibe a janela principal

janela = HorizonPlayer()

janela.show()

# Encerra o interpretador Python de forma limpa ao fechar a janela

sys.exit(app.exec())
```

6. ESTILIZAÇÃO MODERNA COM QSS (style.qss)

O Qt Style Sheets (QSS) permite customizar toda a aparência dos widgets com regras de estilização semelhantes ao CSS da Web. Abaixo está o arquivo style.qss em tons modernos de azul escuro (Dark Slate/Navy) com detalhes em azul elétrico:/* Fundo principal da aplicação */

```
QMainWindow {

    background-color: #0F172A;

    color: #F8FAFC;

}

/* Rótulo de título e faixa atual */

QLabel#label_musica {

    color: #38BDF8;

    font-size: 15px;
```

```
    font-weight: bold;

    padding: 8px;
}

/* Lista de reprodução com scrollbar nativa */

QListWidget {

    background-color: #1E293B;

    border: 1px solid #334155;

    border-radius: 8px;

    color: #E2E8F0;

    padding: 6px;

    font-size: 13px;
}

QListWidget::item {

    padding: 6px;

    border-radius: 4px;
}

QListWidget::item:hover {

    background-color: #334155;
}
```

```
QListWidget::item:selected {  
  
    background-color: #0284C7;  
  
    color: #FFFFFF;  
  
}  
  
/* Barra de rolagem vertical (Scrollbar) personalizada */  
  
QScrollBar:vertical {  
  
    background: #1E293B;  
  
    width: 10px;  
  
    border-radius: 5px;  
  
}  
  
QScrollBar::handle:vertical {  
  
    background: #475569;  
  
    min-height: 20px;  
  
    border-radius: 5px;  
  
}  
  
QScrollBar::handle:vertical:hover {  
  
    background: #0284C7;  
  
}
```



```
/* Botões de controle de reprodução */
```

```
QPushButton {
```

```
    background-color: #1E293B;
```

```
    border: 1px solid #38BDF8;
```

```
    border-radius: 6px;
```

```
    color: #FFFFFF;
```

```
    padding: 6px 12px;
```

```
    font-weight: bold;
```

```
}
```

```
QPushButton:hover {
```

```
    background-color: #0284C7;
```

```
}
```

```
QPushButton:pressed {
```

```
    background-color: #0369A1;
```

```
}
```

```
/* Sliders de progresso e volume */
```

```
QSlider::groove:horizontal {
```

```
    height: 6px;
```

```
    background: #334155;
```

```
border-radius: 3px;

}

QSlider::sub-page:horizontal {

    background: #38BDF8;

    border-radius: 3px;

}

QSlider::handle:horizontal {

    background: #F8FAFC;

    border: 1px solid #38BDF8;

    width: 14px;

    margin-top: -4px;

    margin-bottom: -4px;

    border-radius: 7px;

}
```

7. ATIVIDADES PRÁTICAS E DESAFIOS PARA ALUNOS

Para fixação de conteúdo em laboratório, sugere-se propor aos alunos:

1. **Desafio 1 (Mudo / Mute):** Criar um botão com ícone `QStyle.SP_MediaVolumeMuted` que alterne o volume entre 0% e o valor anterior ao clique.
2. **Desafio 2 (Remover Faixa):** Adicionar um botão "Remover" que exclua o item atualmente selecionado no `QListWidget` e na lista interna `self.playlist`.
3. **Desafio 3 (Limpar Lista):** Criar um método para interromper a reprodução, resetar a interface e esvaziar a fila de reprodução.
4. **Desafio 4 (Modo Repetição / Loop):** Implementar uma caixa de seleção (`QCheckBox`) que, se marcada, reinicie a música atual ou recomece a playlist do início ao término da última faixa.